

Web Development Bootcamp

Frontend Engineering Track — v4.0 Production Curriculum

Curriculum Expansion & Planning Document — For Absolute Beginners

Career Path	Frontend → Full Stack → Agentic AI
Target Audience	Absolute Beginners — Zero Prior Experience
Program Duration	10 Weeks / ~120 Teaching Hours
Modules	10 Core + 4 Portfolio Projects
Version	v4.0 — Production Curriculum

Benchmarked against: Udemy Web Dev Bootcamp · Coursera Meta Frontend Developer · Scrimba Frontend Career Path

SECTION 1 — PROGRAM OVERVIEW & INDUSTRY BENCHMARK

Program Philosophy

doors2ai positions its curriculum at the intersection of beginner-accessibility and production-grade engineering. Unlike pure tutorial platforms, every module targets a concrete skill outcome aligned to the Full Stack → Agentic AI career ladder. The pedagogical sequence — mental model first, then syntax, then applied project — mirrors evidence-based learning design seen in leading bootcamps.

Industry Benchmark Comparison

Dimension	Udemy (Colt Steele)	Coursera (Meta FE)	Scrimba FE Path	doors2ai (This Program)
Format	Video-heavy, solo	Structured, graded	Interactive code editor	Project-driven, mentor-guided
Beginner Fit	★★★★■	★★★██	★★★★★	★★★★★
AI/SaaS Focus	None	None	Minimal	Core to all projects
TypeScript	Optional add-on	Not covered	Partial	Mandatory from Mod 6
Next.js	Basic	Not covered	Not covered	Full module + portfolio
Portfolio Projects	2–3 guided	1 capstone	3 screencasts	4 production builds
Career Bridge	None	Meta ecosystem	Job guarantee option	Full Stack + Agentic AI
Estimated Hours	~65 hrs	~85 hrs	~70 hrs	~120 hrs

Key Differentiator: No comparable curriculum explicitly trains beginners to build UI for AI/SaaS platforms as a stated career outcome. This is doors2ai's primary competitive moat.

SECTION 2 — DETAILED MODULE CURRICULUM PLAN

Module 0 — Environment Setup

Pre-Week / Day 1 | ■ 3–4 hrs

Prerequisites: Computer access only

Learning Objectives

- ✓ Install and configure all essential developer tools
- ✓ Understand the role of each tool in the development workflow
- ✓ Write and run a first HTML file in the browser
- ✓ Navigate VS Code confidently

Subtopics / Topic Breakdown

- What is a browser & how it renders HTML
- Installing VS Code + essential extensions
- Chrome DevTools orientation
- File system organisation for projects
- Git: init, add, commit, push basics
- GitHub account & first repository
- Node.js & npm — why developers need them
- Terminal / command-line basics

Recommended Lesson Sequence

1. Browser tour & DevTools intro
2. Install VS Code + Prettier/ESLint
3. Terminal basics hands-on
4. Create first project folder
5. Install Node.js & verify
6. Git init → commit → GitHub push
7. Open first HTML in browser

Best Practices to Teach

- Keep one project per folder
- Commit early and often
- Name files with lowercase-kebab-case
- Always open DevTools during dev

Common Beginner Mistakes to Cover

- Spaces in file/folder names
- Skipping Git setup ('I'll do it later')
- Using Notepad instead of a code editor
- Fear of the terminal — address directly

Key Concepts to Emphasise

Browser rendering pipeline · Version control mindset · Developer workflow loop

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
• Install checklist verification • Create and open hello-world.html • First Git commit walkthrough • Navigate 5 terminal commands from memory	Dev Environment Checklist — learner documents every tool installed with screenshot proof	• Tool purposes • Basic terminal commands • Git stages (working/staging/committed)	Push a personal bio text file to GitHub

Module 1 — Web Foundations

Week 1 (Days 1–2) | ■ 4–5 hrs

Prerequisites: Module 0 complete

Learning Objectives

- ✓ Explain how the internet delivers web pages
- ✓ Describe the client-server model in plain language
- ✓ Differentiate HTML, CSS, and JavaScript responsibilities
- ✓ Understand what browsers do with code

Subtopics / Topic Breakdown	
• What is the internet vs. the Web	Recommended Lesson Sequence • 1. Internet vs. Web analogy • 2. URL anatomy walkthrough • 3. HTTP request demo in DevTools • 4. HTML/CSS/JS role separation diagram • 5. DOM concept introduction • 6. Static vs. dynamic comparison • 7. Framework motivation teaser
• DNS, IP addresses, HTTP/HTTPS	
• Client-server model	
• Request-response cycle	
• What browsers do (parsing, rendering, painting)	
• HTML / CSS / JS roles — separation of concerns	
• Introduction to the DOM concept	
• Static vs. dynamic websites	
• Introduction to frameworks (why they exist)	
Best Practices to Teach	
• Always separate structure (HTML) from style (CSS)	Common Beginner Mistakes to Cover • Confusing the internet with the browser • Thinking JavaScript is Java • Skipping the mental model — diving into code too fast
• Think in layers: content → presentation → behaviour	
• Use DevTools Network tab to inspect requests	

Key Concepts to Emphasise

Separation of concerns · DOM · Request-response cycle · Rendering pipeline

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
• Draw the request-response cycle from memory • Inspect a live website in DevTools Network tab • Label HTML/CSS/JS on a screenshot of any website • Explain DNS to a partner in 60 seconds	Web Anatomy Map — annotated diagram of how a chosen website's layers (HTML/CSS/JS) interact	• HTTP vs HTTPS • DOM definition • Role of each web technology • Client vs. server	Research and write a short explanation of what happens when you type a URL and press Enter

Module 2 — HTML

Week 1 (Days 2–5) | ■ 10–12 hrs

Prerequisites: Module 1 — Web Foundations

Learning Objectives

- ✓ Write valid, semantic HTML5 documents from scratch

- ✓ Understand the purpose of every major HTML element
- ✓ Build accessible page structures using ARIA roles
- ✓ Create forms with correct input types and validation attributes
- ✓ Link pages together and embed media

Subtopics / Topic Breakdown	
• HTML document anatomy (DOCTYPE, html, head, body)	Recommended Lesson Sequence • 1. Boilerplate anatomy line by line • 2. Block vs inline demo • 3. Semantic element page structure • 4. Heading hierarchy + SEO context • 5. Links and navigation • 6. Images + alt text importance • 7. Forms from scratch • 8. Input types exploration • 9. Tables with accessible markup • 10. Meta tags and social preview • 11. Accessibility audit intro
• Block vs. inline elements	
• Semantic HTML5 elements (header, nav, main, section, article, footer)	
• Headings hierarchy (h1–h6) and SEO implications	
• Paragraphs, lists (ul/ol/dl), blockquote	
• Hyperlinks — absolute, relative, anchor	
• Images: src, alt, width/height, lazy loading	
• Tables: when to use and accessibility	
• HTML Forms: inputs, labels, fieldsets, select, textarea	
• Input types and HTML5 validation attributes	
• HTML entities and special characters	
• Meta tags: charset, viewport, description, og tags	
• Embedding video and audio	
• Introduction to accessibility and ARIA	
Best Practices to Teach	
• Always include alt text on images	Common Beginner Mistakes to Cover • Using the br tag for spacing instead of CSS
• One h1 per page	• Nesting block elements inside inline elements
• Use semantic elements over generic divs	• Skipping the doctype declaration
• Associate every label with its input	• Missing closing tags
• Never use tables for layout	• Using heading tags for visual size not hierarchy • Forgetting name attribute on form inputs

Key Concepts to Emphasise

Semantic HTML · Accessibility · Document structure · Form validation · Progressive enhancement

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
------------------------------	--------------	-------------	----------

• Build a personal bio page using only semantic HTML • Recreate a given wireframe in HTML only • Build a multi-field contact form • Add a navigation menu with anchor links • Fix the broken HTML challenge (5 planted bugs)	Personal Profile Page — multi-section HTML-only page with nav, bio, projects list, and contact form	• Semantic vs non-semantic • Form input types • Accessibility attributes • Meta tags purpose	Build an HTML resume page without any CSS
--	---	---	---

Module 3 — CSS

Week 2 | ■ 14–16 hrs

Prerequisites: Module 2 — HTML

Learning Objectives

- ✓ Apply styles using selectors, properties, and values
- ✓ Master the box model and layout techniques
- ✓ Build responsive layouts with Flexbox and CSS Grid
- ✓ Use CSS custom properties and basic animations
- ✓ Implement a mobile-first responsive design approach

Subtopics / Topic Breakdown

- CSS syntax: selectors, properties, values
- Cascade, specificity, and inheritance
- Box model: margin, border, padding, content
- Display types: block, inline, inline-block, none
- Positioning: static, relative, absolute, fixed, sticky
- Flexbox — all properties, alignment, wrapping
- CSS Grid — template areas, fr unit, auto-fill
- CSS custom properties (variables)
- Responsive design & media queries
- Mobile-first approach
- Typography: font-family, size, weight, line-height
- Colors: hex, rgb, hsl, opacity
- Pseudo-classes and pseudo-elements
- CSS transitions and animations
- CSS resets and normalise
- Utility-first CSS overview (Tailwind preview)

Recommended Lesson Sequence

- 1. Linking CSS to HTML, syntax basics
- 2. Selectors deep-dive
- 3. Box model visualised in DevTools
- 4. Positioning challenge
- 5. Flexbox module (nav, cards, centering)
- 6. CSS Grid module (layouts)
- 7. CSS variables system
- 8. Media queries, mobile-first
- 9. Typography and color systems
- 10. Pseudo-classes and hover states
- 11. Transitions and keyframe animations
- 12. Responsive page audit

Best Practices to Teach	Common Beginner Mistakes to Cover
• Use CSS variables for colour and spacing tokens	• Using px for everything instead of rem/em
• Mobile-first: write base styles for small screens first	• Overusing absolute positioning
• Prefer Flexbox/Grid over float or positioning hacks	• Ignoring the cascade — fighting it with !important
• Keep specificity low — avoid !important	• Not testing on multiple screen sizes
• Organise CSS: reset → base → layout → components	• Writing CSS without a consistent naming system
	• Using margin:auto without display:block

Key Concepts to Emphasise

Box model · Cascade & specificity · Flexbox · CSS Grid · Mobile-first · Custom properties

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
• Holy Grail Layout — Flexbox only • Responsive card grid — CSS Grid • Navigation bar with hover animations • Recreate a given UI screenshot in CSS • Fix the broken layout challenge • Build a CSS colour theme with custom properties	Styled Profile Page — take Module 2 HTML and apply full professional CSS styling with responsive layout	• Specificity calculation • Flexbox vs Grid use cases • Box model properties • Media query breakpoints	Clone a simple landing page design (Figma provided) using HTML + CSS

Module 4 — JavaScript Foundations

Week 3 | ■ 16–18 hrs

Prerequisites: Modules 2 & 3 — HTML + CSS

Learning Objectives

- ✓ Write JavaScript to manipulate the DOM
- ✓ Understand variables, data types, control flow, and functions
- ✓ Handle user events and form input
- ✓ Debug JavaScript errors confidently in DevTools
- ✓ Understand scope, closures, and the call stack at a conceptual level

Subtopics / Topic Breakdown
• Where JS runs: browser vs Node.js
• Variables: var/let/const — differences
• Data types: string, number, boolean, null, undefined, object, array
• Type coercion and strict equality
• Control flow: if/else, ternary, switch
• Loops: for, while, for...of, for...in
• Functions: declaration, expression, arrow functions
• Scope: global, function, block
• Arrays: methods (map, filter, reduce, find)
• Objects: creation, access, methods
• DOM selection and manipulation
• Event listeners and event object
• Template literals
• Basic error handling: try/catch
• Debugging with console and DevTools

Best Practices to Teach
• Use const by default, let when reassignment needed
• Prefer === over ==
• Name functions with verbs (handleClick, fetchData)
• Keep functions small and single-purpose
• Console.log strategically, then remove

Recommended Lesson Sequence
• 1. Connecting JS to HTML, console.log
• 2. Variables and data types
• 3. Operators and type coercion
• 4. Control flow — if/else, switch
• 5. Loops — practical exercises
• 6. Functions — declaration to arrow
• 7. Scope visualised
• 8. Arrays + higher-order methods
• 9. Objects and JSON preview
• 10. DOM manipulation hands-on
• 11. Event listeners — click, input, submit
• 12. Forms + JS validation
• 13. Debugging workshop

Common Beginner Mistakes to Cover
• Using var and confusing hoisting
• == vs === confusion
• Mutating arrays without understanding
• Forgetting to add event listeners after DOM loads
• Not reading error messages — skipping to StackOverflow
• Monolithic functions doing too many things

Key Concepts to Emphasise

Event-driven programming · DOM manipulation · Functions as first-class values · Scope chain · Array methods

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
------------------------------	--------------	-------------	----------

• Build a counter with increment/decrement buttons • - To-do list: add and remove items from DOM • - Form validator with live error messages • - Array transformation challenge (map/filter/reduce) • - Colour picker that updates page background	Interactive Quiz App — 5 questions, score tracking, show/hide results, restart button	• var/let/const scoping • - Event bubbling • - Array methods • - DOM selection methods • - Type coercion	Build a tip calculator with form inputs and real-time output
--	---	--	--

Module 5 — Modern JavaScript (ES6+)

Week 4 | ■ 10–12 hrs

Prerequisites: Module 4 — JavaScript Foundations

Learning Objectives

- ✓ Write asynchronous JavaScript using Promises and async/await
- ✓ Fetch data from public APIs and render it in the DOM
- ✓ Use ES6+ features: destructuring, spread, modules
- ✓ Understand the event loop and why async matters
- ✓ Handle API errors gracefully

Subtopics / Topic Breakdown

- ES6 review: arrow functions, template literals, defaults
- Destructuring: arrays and objects
- Rest and spread operators
- Short-circuit evaluation and optional chaining
- Nullish coalescing
- ES Modules: import/export
- Promises: .then/.catch/.finally
- async/await syntax
- Fetch API and JSON
- Error handling in async code
- The event loop explained simply
- LocalStorage and SessionStorage
- Array methods: flat, flatMap, Object.entries/keys/values

Recommended Lesson Sequence

- 1. ES6 syntax refresher
- 2. Destructuring deep-dive
- 3. Spread/rest practical uses
- 4. Event loop mental model
- 5. Promises — building from scratch
- 6. async/await refactor
- 7. Fetch API — GET request
- 8. Render API data to DOM
- 9. Error states (network, 404, parsing)
- 10. ES Modules organisation
- 11. LocalStorage persistence

Best Practices to Teach

- Always handle async errors with try/catch
- Never nest Promises — use async/await
- Show loading and error states in UI
- Separate API calls from DOM rendering logic

Common Beginner Mistakes to Cover

- Forgetting await — getting a Promise object instead of data
- Not handling fetch errors (fetch doesn't reject on 404)
- Blocking the main thread with synchronous operations
- Circular imports in module systems

Key Concepts to Emphasise

Asynchronous programming · Event loop · Promises · Fetch API · ES Modules · Optional chaining

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
• Fetch and display GitHub user profile • Weather card using OpenWeather API • Refactor Promise chain to async/await • LocalStorage bookmarks manager • Error boundary simulation exercise	Country Explorer — search any country, fetch REST Countries API, display flag, population, languages, region	• Event loop explanation • async/await vs Promises • Fetch error handling • Destructuring syntax	Build a movie search app using OMDb API with loading and error states

Module 6 — TypeScript

Week 5 | ■ 10–12 hrs

Prerequisites: Module 5 — Modern JavaScript

Learning Objectives

- ✓ Understand why TypeScript exists and what problems it solves
- ✓ Annotate variables, functions, and objects with types
- ✓ Use interfaces and type aliases to model data
- ✓ Work with generics at a basic level
- ✓ Configure a TypeScript project with tsconfig

Subtopics / Topic Breakdown
• What is TypeScript and why it matters for teams
• Type inference vs explicit annotation
• Primitive types: string, number, boolean, null, undefined
• Arrays and tuples
• Union types and type narrowing
• Interfaces vs type aliases
• Optional and readonly properties
• Function parameter and return types
• Enums
• Generics — basic introduction
• Type assertions
• tsconfig.json essentials
• TypeScript with DOM APIs
• Utility types: Partial, Required, Pick, Omit

Recommended Lesson Sequence
• 1. TypeScript motivation and overview
• 2. Installation and tsconfig basics
• 3. Annotating primitives and arrays
• 4. Function types
• 5. Interfaces for objects
• 6. Union types and narrowing
• 7. Generics introduction
• 8. Type utilities
• 9. Converting a JS module to TS
• 10. TypeScript + DOM exercise

Best Practices to Teach
• Prefer interfaces for object shapes, type for unions/aliases
• Enable strict mode in tsconfig
• Avoid any — treat it as a warning signal
• Use explicit return types on public functions

Common Beginner Mistakes to Cover
• Using any everywhere — defeating the purpose
• Confusing interface and type syntax
• Not understanding type narrowing
• Ignoring tsconfig strict flags

Key Concepts to Emphasise

Static typing · Type inference · Interfaces · Generics · strict mode · Utility types

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
- Type a set of given JavaScript functions - Model an API response with interfaces - Refactor Module 5 fetch app to TypeScript - Generic utility function exercise - Fix the TypeScript errors challenge (10 errors planted)	Typed Todo App — full TS todo manager with typed state, events, and localStorage persistence	- Interface vs type - Union type definition - any vs unknown - Generic syntax - tsconfig strict	Convert Module 4 quiz app to TypeScript with full type coverage

Module 7 — React Foundations

Week 6 | ■ 14–16 hrs

Prerequisites: Module 6 — TypeScript

Learning Objectives

- ✓ Explain the component model and virtual DOM
- ✓ Build functional components with JSX
- ✓ Manage local state with useState
- ✓ Handle side effects with useEffect
- ✓ Pass and type props between components
- ✓ Build a multi-component application

Subtopics / Topic Breakdown	
• Why React exists — problems it solves	
• Virtual DOM and reconciliation (mental model)	
• JSX syntax and transpilation	
• Functional components	
• Props — passing data down	
• useState — reading and updating state	
• Conditional rendering	
• List rendering and keys	
• Event handling in React	
• useEffect — side effects and cleanup	
• Controlled inputs	
• Component composition and children prop	
• Lifting state up	
• Basic component file structure	
• Intro to React DevTools	

Recommended Lesson Sequence	
• 1. React mental model — why components	
• 2. Create React app with Vite + TS	
• 3. First component and JSX	
• 4. Props — parent to child	
• 5. useState — counter exercise	
• 6. Conditional rendering patterns	
• 7. List rendering + key prop	
• 8. Event handlers	
• 9. Controlled forms	
• 10. useEffect — fetch on mount	
• 11. Lifting state up	
• 12. Component composition	
• 13. React DevTools tour	

Best Practices to Teach	
• One component per file	
• Keep components small and focused	
• State belongs as close to where it's used as possible	
• Always provide keys for list items	
• Clean up useEffect subscriptions	

Common Beginner Mistakes to Cover	
• Mutating state directly instead of using setter	
• Missing keys in list rendering	
• Calling hooks conditionally	
• Fetching inside render instead of useEffect	
• Over-lifting state (prop drilling too early)	

Key Concepts to Emphasise

Component model · Virtual DOM · Unidirectional data flow · State immutability · Side effects lifecycle

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
------------------------------	--------------	-------------	----------

• Traffic light component with state cycling • Searchable list with controlled input • Fetch and display API data in <code>useEffect</code> • Accordion component with open/close state • Shopping cart with add/remove and total	Recipe Browser — search recipes API, display cards, click to expand details, favourite toggle with local state	• <code>useState</code> update rules • <code>useEffect</code> dependency array • Props vs state • Key prop purpose • Controlled vs uncontrolled	Build a multi-step form with React state management
---	--	---	---

Module 8 — Intermediate React

Week 7 | ■ 14–16 hrs

Prerequisites: Module 7 — React Foundations

Learning Objectives

- ✓ Manage complex state with `useReducer` and `Context`
- ✓ Build reusable custom hooks
- ✓ Optimise performance with `useMemo`, `useCallback`, `React.memo`
- ✓ Implement React Router for client-side navigation
- ✓ Structure a mid-scale React application professionally

Subtopics / Topic Breakdown

- `useReducer` for complex state
- `Context` API — creating, providing, consuming
- `Context` + `useReducer` pattern (mini `Redux`)
- Custom hooks — extracting and reusing logic
- React Router v6 — routes, links, params
- Nested routing and layouts
- URL query parameters
- `useMemo` and `useCallback`
- `React.memo` for component memoisation
- Code splitting with `React.lazy` and `Suspense`
- Error boundaries
- Forms with React Hook Form (intro)
- Component patterns: render props, compound components
- Folder structure conventions at scale

Recommended Lesson Sequence

- 1. `useReducer` refactor from `useState`
- 2. `Context` API setup and consumption
- 3. `Context` + reducer global store
- 4. Custom hooks extraction
- 5. React Router — basic routing
- 6. Dynamic routes and params
- 7. Nested layouts
- 8. Performance tools intro
- 9. `useMemo`/`useCallback` scenarios
- 10. `React.memo` demonstration
- 11. `Suspense` + lazy loading
- 12. Error boundaries
- 13. Folder architecture workshop

Best Practices to Teach	Common Beginner Mistakes to Cover
• Keep Context small and specific — avoid one god context	• Overusing Context for everything
• Extract custom hooks for any reused logic	• Creating custom hooks that wrap a single useState
• Memoize only when profiler proves a bottleneck	• Premature optimisation with useMemo
• Use React Router data APIs for loading states	• Not separating routing from layout components

Key Concepts to Emphasise

State architecture · Context API · Custom hooks · Client-side routing · Performance profiling

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
• Refactor Module 7 project to use useReducer • Theme switcher using Context • Build a useFetch custom hook • Multi-page React Router app with nav • Performance profiling exercise — find and fix re-renders	Team Dashboard — multiple pages (members, projects, settings), global auth context, protected routes	• useReducer action pattern • Context re-render behaviour • Router hooks: useParams, useNavigate • Memo vs useMemo	Refactor an existing app to use custom hooks and Context-based theme

Module 9 — Next.js

Week 8 | ■ 14–16 hrs

Prerequisites: Module 8 — Intermediate React

Learning Objectives

- ✓ Understand when and why to use Next.js over plain React
- ✓ Use the App Router with layouts, pages, and loading states
- ✓ Implement Server Components and Client Components correctly
- ✓ Fetch data using server-side patterns
- ✓ Deploy a Next.js application to Vercel

Subtopics / Topic Breakdown
• What is Next.js and its rendering models (SSR, SSG, ISR, CSR)
• App Router vs Pages Router
• File-based routing and route segments
• Layouts and nested layouts
• Server Components vs Client Components
• Loading UI and Suspense boundaries
• Error.tsx handling
• Route Handlers (API routes)
• Data fetching: fetch with caching in Server Components
• Dynamic routes and generateStaticParams
• Metadata API (SEO)
• Image component optimisation
• Next.js fonts
• Environment variables
• Deploying to Vercel

Recommended Lesson Sequence
• 1. Next.js motivation — SSR/SSG mental model
• 2. Create Next.js project, folder tour
• 3. App Router routing basics
• 4. Layouts — root and nested
• 5. Server vs Client components demo
• 6. Data fetching in Server Components
• 7. Loading.tsx and Suspense
• 8. Error.tsx boundaries
• 9. Dynamic routes
• 10. Route Handlers (API layer)
• 11. Metadata and SEO
• 12. Image and font optimisation
• 13. Environment variables
• 14. Deploy to Vercel walkthrough

Best Practices to Teach
• Default to Server Components — add 'use client' only when needed
• Co-locate loading.tsx with each route segment
• Never expose secrets in client components
• Use Next/Image for all images

Common Beginner Mistakes to Cover
• Adding 'use client' to every component by default
• Fetching data in client components when server would do
• Forgetting that Server Components can't use hooks/events
• Hardcoding API keys in component files

Key Concepts to Emphasise

Rendering strategies · App Router · Server Components · Streaming · Vercel deployment · Metadata API

Suggested Exercises (Titles)	Mini-Project	Quiz Topics	Homework
• Blog with dynamic [slug] routes and SSG • API route that returns JSON data • Nested layout with shared sidebar • Image gallery with Next/Image optimisation • Deploy Module 8 project to Vercel	Product Catalogue — SSG product listing, dynamic product detail pages, metadata, deployed to Vercel	• SSR vs SSG vs ISR • Server vs Client component rules • App Router file conventions • fetch caching options	Build a portfolio site in Next.js and deploy live to Vercel

SECTION 3 — PORTFOLIO PROJECT PLANNING

Each portfolio project is a complete, production-quality build. Planning details below include description, skills covered, learning purpose, complexity level, and suggested build phases. No code or solutions are provided here.

Project 1: Professional Portfolio Website

Complexity: Intermediate — 3 to 5 days

Learning Purpose

Validates HTML/CSS mastery and demonstrates the learner can build a professional online presence. This is the learner's first live, public-facing project — a key confidence milestone. Directly comparable to Scrimba's Career Path milestone project.

Skills Covered

HTML5 Semantics · CSS Grid + Flexbox · Responsive Design · React · Next.js · TypeScript · Vercel Deploy

Suggested Build Phases

- Phase 1: Wireframe and content planning — define sections and copy
- Phase 2: HTML structure — semantic, accessible skeleton
- Phase 3: CSS styling — design system, variables, typography, responsive layout
- Phase 4: React migration — convert to component architecture
- Phase 5: Next.js conversion — metadata, SSG, image optimisation
- Phase 6: TypeScript integration — type all props and data
- Phase 7: Polish — animations, accessibility audit, performance check
- Phase 8: Deploy to Vercel — live URL for portfolio

Project 2: AI Product Dashboard UI

Complexity: Advanced — 5 to 7 days

Learning Purpose

Simulates real-world AI platform UI engineering — the type of work found at companies building LLM products, analytics platforms, and AI tooling. Learners build the kind of UI that a junior frontend engineer would be assigned at a SaaS or AI startup. No comparable project exists in Udemy, Coursera, or Scrimba beginner tracks.

Skills Covered

React Architecture · Component Design System · Data Tables · Filters & Search UI · Charts (Recharts) · TypeScript · Context API

Suggested Build Phases

- Phase 1: Design system setup — colour tokens, typography scale, spacing system
- Phase 2: Layout shell — sidebar nav, header, main content area
- Phase 3: Dashboard overview — metric cards, summary stats
- Phase 4: Data table — sortable, filterable, paginated
- Phase 5: Charts module — line/bar charts with Recharts, mock data

Phase 6: Filter and search UI — live filtering, tag filters

Phase 7: Responsive adaptation — mobile drawer nav, stacked layout

Phase 8: TypeScript audit — full type coverage of all data and props

Project 3: SaaS Frontend Application

Complexity: Advanced — 6 to 8 days

Learning Purpose

Demonstrates full Next.js application architecture — the most directly employable skill in the React/Next.js job market. Learners build a multi-section SaaS shell that mirrors real products (think Vercel, Linear, or Notion settings UX). Prepares learners for Full Stack module by showing clear frontend/backend separation.

Skills Covered

Next.js App Router · Nested Layouts · Settings UI · Billing UI · Feature Pages · Server Components · Route Handlers · TypeScript

Suggested Build Phases

Phase 1: App architecture — route planning, layout hierarchy, folder structure

Phase 2: Root layout — nav, sidebar, auth-aware wrapper

Phase 3: Feature pages — dashboard home, feature overview, usage stats

Phase 4: Settings section — profile, notifications, security settings pages

Phase 5: Billing UI — plan cards, usage meter, upgrade flow UI

Phase 6: Server Components + data fetching — server-side rendering with mock API

Phase 7: API Route Handlers — settings save endpoints

Phase 8: Loading, error, and empty states for all pages

Phase 9: Deploy and audit — Vercel deploy, Lighthouse performance check

Project 4: Learning Platform UI

Complexity: Capstone — 7 to 10 days

Learning Purpose

Capstone project — integrates every skill from the program into the most complex build. A learning platform UI is directly relevant to the doors2ai product ecosystem and demonstrates readiness for Full Stack and Agentic AI engineering. This project is the centrepiece portfolio piece for job applications.

Skills Covered

React · Next.js · Course Dashboard · Lesson Viewer · Progress Tracking UI · Student Dashboard · Responsive Design · TypeScript

Suggested Build Phases

Phase 1: Product definition — map all screens, user flows, and component inventory

Phase 2: Design system — tokens, component library (Button, Card, Badge, Progress)

Phase 3: Student dashboard — enrolled courses, progress overview, recent activity

Phase 4: Course catalogue — search, filter by category, course cards

Phase 5: Course detail page — syllabus, instructor info, enrol CTA

Phase 6: Lesson viewer — sidebar nav, video placeholder, lesson content

Phase 7: Progress UI — completion bars, chapter checkmarks, certificate unlock state

Phase 8: Responsive design pass — mobile lesson viewer, bottom nav

Phase 9: TypeScript full coverage + code quality review

Phase 10: Deploy + portfolio documentation — README, live URL, feature walkthrough

SECTION 4 — ASSESSMENT STRUCTURE & WEEKLY SCHEDULE

Assessment Framework

Assessment Type	When	Weight	Evaluation Criteria
Module Quiz (MCQ + short answer)	End of each module	10%	Concept accuracy, mental model clarity
Exercises (in-class)	During each module	15%	Completion, correct output, code quality
Mini-Projects	End of each module	20%	Feature completeness, code organisation
Portfolio Projects (4 total)	Weeks 6–10	40%	UI quality, component design, TS coverage
Homework Assignments	Weekly	15%	Effort, progression, self-directed learning

Final Project Evaluation Rubric

Criterion	Excellent (4)	Good (3)	Needs Work (2)	Incomplete (1)
Project Quality	Production-grade, polished	Functional, minor issues	Works but rough	Broken or incomplete
Code Clarity	Clean, named, modular	Mostly clean	Hard to follow	No structure
UI Quality	Professional, accessible	Decent design	Functional but unstyled	Poor/broken UI
Component Design	Reusable, well-composed	Some reuse	Monolithic	No components
TypeScript Usage	Full coverage, no any	Mostly typed	Partial typing	Not used
Problem Solving	Independent, methodical	Mostly independent	Needed significant help	Unable to proceed

10-Week Delivery Schedule

Week	Module(s)	Key Deliverable	Hrs	Benchmark Parity
Week 1	Mod 0 + 1 + 2 (HTML)	HTML Profile Page	14–16	Udemy ✓ · Scrimba ✓
Week 2	Mod 3 (CSS)	Styled Responsive Page	14–16	All bootcamps ✓
Week 3	Mod 4 (JavaScript)	Interactive Quiz App	16–18	Udemy ✓ · Coursera ✓
Week 4	Mod 5 (Modern JS)	Country Explorer App	12–14	Scrimba ✓
Week 5	Mod 6 (TypeScript)	Typed Todo App	10–12	doors2ai advantage
Week 6	Mod 7 (React Foundations)	Recipe Browser App	14–16	All bootcamps ✓

Week 7	Mod 8 + Portfolio P1 & P2	Portfolio + Dashboard	16–18	doors2ai advantage
Week 8	Mod 9 (Next.js)	Product Catalogue Live	14–16	doors2ai advantage
Week 9	Portfolio P3 (SaaS UI)	Live SaaS Frontend	16–18	doors2ai advantage
Week 10	Portfolio P4 (Learning Platform)	Capstone Deploy + Review	16–18	doors2ai advantage

SECTION 5 — INSTRUCTOR GUIDANCE & CAREER BRIDGE

Instructor Teaching Principles

DO — Recommended Instructor Behaviours

- Explain the mental model before showing code
- Predict-before-run: ask learners what they expect
- Demonstrate debugging live — make it normal to hit errors
- Celebrate incremental wins — build confidence
- Move slowly through fundamentals — speed up with frameworks
- Encourage questions before Googling
- Revisit earlier concepts in later modules (spiral learning)

DO NOT — Common Instructor Pitfalls

- Rush through HTML/CSS to get to React
- Assume any prior knowledge without checking
- Skip error messages — just give the answer
- Introduce multiple new concepts in one session
- Overwhelm with tooling before core concepts are solid
- Make learners feel behind if they struggle

Career Outcomes & Readiness Checklist

Outcome	Evidence in Portfolio	Next Step Alignment
Build professional frontend apps	Projects 1–4	Junior FE role or Full Stack program
React + Next.js proficiency	Projects 2–4 + Module 9	Full Stack Engineering
Build UI for AI platforms	Project 2 (AI Dashboard)	Agentic AI Engineering program
TypeScript usage	All Projects (typed)	Production team environments
API integration	Modules 5, 8, 9 + Projects	Backend and API-first roles
Responsive, accessible UI	All Projects	UX-aware engineering roles

Bridge to Full Stack + Agentic AI Program

Upon completion of this Frontend Engineering track, learners are prepared to enter the Full Stack Development program. The bridge curriculum will cover: Node.js & Express, REST API design, databases (PostgreSQL/MongoDB), authentication, and API integration from the backend perspective. Graduates of Full Stack then enter the Agentic AI Engineering program, where they build AI-powered applications, integrate LLM APIs (Anthropic, OpenAI), design agent architectures, and build the production UI they already know how to create.

Step 1 Frontend Engineering
(This Program)



Step 2 Full Stack Development
(Next Program)



Step 3 Agentic AI Engineering
(Final Track)

